



TRABALHO DE CONCLUSÃO DE CURSO
UNIVERSIDADE FEDERAL DE ITAJUBÁ - *Campus de Itabira*
INSTITUTO DE CIÊNCIAS TECNOLÓGICAS
ENGENHARIA DE COMPUTAÇÃO

**SOFTWARE DE GESTÃO DE INFORMAÇÃO PERSONALIZÁVEL UTILIZANDO ARQUITETURA
LIMPA E APLICAÇÃO DE PRINCÍPIOS S.O.L.I.D**

THALLES AUGUSTO MONTEIRO MARTINS, EDUARDO RIBEIRO FELIPE*, RODRIGO APARECIDO DA SILVA
BRAGA †

** Universidade Federal de Itajubá - Campus de Itabira
Rua Irmã Ivone Drumond, 200 - Distrito Industrial II - 35903-087
Itabira, Minas Gerais, Brasil*

*† Universidade Federal de Lavras
Trevo Rotatório Professor Edmir Sá Santos - 37203-202
Lavras, Minas Gerais, Brasil*

E-mails: thalles@unifei.edu.br, eduardo.felipe@unifei.edu.br, rodrigobraga@ufla.br

Resumo— Este trabalho apresenta o desenvolvimento de um *software* de gestão da informação que, embora semelhante a ferramentas de gestão da informação existentes, oferece maior liberdade de personalização aos usuários. Utilizando os princípios da Arquitetura Limpa e incorporando técnicas modernas de desenvolvimento de *software* com boas práticas de desenvolvimento e aplicação de princípios S.O.L.I.D, o sistema proposto visa proporcionar alta flexibilidade e robustez. Essa abordagem permite que tanto os usuários finais quanto os desenvolvedores responsáveis pela manutenção e implementação de novas funcionalidades beneficiem-se de um ambiente adaptável e sustentável, alinhado com os princípios de design que promovem escalabilidade e facilidade de manutenção.

Palavras-chave— gestão da informação, arquitetura limpa, boas práticas de desenvolvimento, S.O.L.I.D, flexibilidade, robustez, desenvolvimento de *software*, sistemas adaptáveis

1 Introdução

Na era digital, o domínio da *gestão da informação* tornou-se decisivo para que organizações e indivíduos convertam grandes volumes de dados em decisões estratégicas (M. E. Porém; V. C. Bissoli; R. C. Belluzzo, 2012; McKnight, 2013). Metodologias ágeis e fluxos automatizados reforçam essa capacidade ao promover ciclos iterativos, colaboração entre equipes e transparência operacional, elevando a eficiência interna (R. A. C. Júnior, T. S. Nunes, 2021).

Paralelamente, o cenário atual de desenvolvimento de *software* é profundamente influenciado por ferramentas assistidas por inteligência artificial, que aceleram a geração de código, mas também incentivam a reutilização acrítica de soluções prontas. Neste contexto, torna-se ainda mais relevante o uso de uma arquitetura bem fundamentada no processo de desenvolvimento. Ao invés de depender exclusivamente de trechos de código de terceiros, o programador passa a organizar o sistema com base em princípios sólidos, favorecendo a manutenção, segurança e coerência da solução como um todo.

Este artigo apresenta o desenvolvimento de um

software de gestão da informação personalizável, permitindo que o usuário configure suas próprias bases de dados, adequando a ferramenta aos seus processos em vez de moldar processos ao sistema. Para alcançar tal flexibilidade, adotam-se os princípios da Arquitetura Limpa e as boas práticas S.O.L.I.D (acrônimo para *Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation e Dependency Inversion*), que fomentam modularidade, independência de tecnologias externas e escalabilidade (Erich Gamma, 1994; Martin, 2019). Dessa forma, o núcleo de regras de negócio mantém-se isolado de detalhes de implementação, favorecendo testes, manutenções e futuras evoluções — atributos essenciais num ambiente corporativo sujeito a rápidas mudanças (McKnight, 2013).

2 Fundamentação Teórica

Esta seção aborda a fundamentação teórica sobre as linhas de interesse desse manuscrito, sendo assim, a abordagem dos conceitos apresentados é de suma importância para a compreensão do projeto.

A Figura 2 ilustra a estrutura da Arquitetura Limpa, que enfatiza uma separação rigorosa e clara entre as diversas camadas do *software*. No centro, temos a camada *Entities* (Entidades), que representam as regras de negócio do domínio. Circundando as entidades, encontram-se a camada de *Use Cases* (Casos de Uso), que encapsulam as regras de aplicação e definem as operações específicas que podem ser realizadas dentro do sistema.

Avançando para a próxima camada, temos os Adaptadores de Interface (*Interface Adapters*), que incluem Controladores (*Controllers*) e Apresentadores (*Presenters*). Esta camada serve como uma ponte entre as regras de negócio e as operações externas, transformando dados entre os formatos mais convenientes para os casos de uso e para os métodos de entrega externos.

Finalmente, a camada mais externa é composta por Agentes Externos, que incluem dispositivos, web, interface de usuário e *gateways* de banco de dados. Esta camada se encarrega de detalhes como a comunicação com bancos de dados e outros dispositivos ou redes. O fluxo de controle é demonstrado pelo movimento do uso externo, passando pelos apresentadores e controladores, até chegar aos casos de uso e entidades.

A utilização da Arquitetura Limpa no desenvolvimento do projeto é justificada pela necessidade de um *software* escalável e flexível. Em ambientes organizacionais, a implementação de novas tecnologias e a integração de novos *softwares* são comuns, criando a demanda por sistemas que se adaptem a diferentes condições. Portanto, a Arquitetura Limpa oferece essa flexibilidade sem interferir na lógica fundamental do sistema, assegurando que alterações externas não comprometam o núcleo do *software*.

2.3 Boas Práticas de Desenvolvimento

As boas práticas de desenvolvimento de *software* englobam um conjunto de princípios, técnicas e metodologias que visam aprimorar a qualidade do código, facilitar a manutenção e promover a eficiência no processo de criação de sistemas. Entre essas práticas, destacam-se a utilização de padrões de projeto, a aplicação dos princípios S.O.L.I.D, a escrita de código limpo, a realização de testes automatizados e a adoção de metodologias ágeis (Martin, 2009)(Martin, 2019).

A aplicação dessas práticas é fundamental para o desenvolvimento de sistemas robustos e flexíveis, capazes de evoluir conforme as necessidades dos usuários e do mercado. O código limpo, por exemplo, caracteriza-se pela simplicidade, clareza e ausência de duplicações, facilitando o entendimento e a colaboração entre desenvolvedores (Martin, 2009).

No contexto da Arquitetura Limpa, as boas práticas de desenvolvimento desempenham um papel crucial. A Arquitetura Limpa preconiza a independência entre as camadas do sistema, a separação de responsabilidades e a inversão de dependências—princípios

intrinsecamente ligados às boas práticas de programação (Martin, 2019).

2.4 S.O.L.I.D

O acrônimo S.O.L.I.D representa um conjunto de cinco princípios de design orientado a objetos que visam tornar o código mais compreensível, flexível e fácil de manter. Esses princípios foram popularizados por Robert C. Martin, também conhecido como Uncle Bob, e são fundamentais para o desenvolvimento de *software* de alta qualidade (Ugonna Thelma, 2020). Os princípios S.O.L.I.D são:

- **S - Single Responsibility Principle (Princípio da Responsabilidade Única):** Uma classe deve ter apenas um motivo para mudar, ou seja, deve possuir uma única responsabilidade. Isso significa que cada classe ou módulo deve focar em uma única parte da funcionalidade do *software*, evitando acúmulo de responsabilidades que podem tornar o código complexo e difícil de manter.
- **O - Open/Closed Principle (Princípio Aberto/Fechado):** As entidades de *software* (classes, módulos, funções, etc.) devem ser abertas para extensão, mas fechadas para modificação. Isso implica que o comportamento de um módulo pode ser estendido sem alterar seu código fonte original, promovendo a reutilização e minimizando o risco de introduzir novos bugs em código já testado.
- **L - Liskov Substitution Principle (Princípio da Substituição de Liskov):** Objetos de uma classe derivada devem poder ser substituídos por objetos da classe base sem alterar a corretude do programa. Em outras palavras, as subclasses devem ser substituíveis pelas suas superclasses, garantindo que o sistema continue funcionando corretamente quando as classes derivadas são usadas no lugar das classes base.
- **I - Interface Segregation Principle (Princípio da Segregação de Interface):** Muitas interfaces específicas são melhores do que uma interface geral. Isso significa que os clientes não devem ser forçados a depender de interfaces que não utilizam. Ao dividir interfaces grandes em interfaces menores e mais específicas, evitamos que as classes implementem métodos desnecessários, tornando o código mais limpo e fácil de entender.
- **D - Dependency Inversion Principle (Princípio da Inversão de Dependência):** Módulos de alto nível não devem depender de módulos de baixo nível; ambos devem depender de abstrações. Além disso, as abstrações não devem depender de detalhes; os detalhes devem depender das abstrações. Esse princípio promove um baixo acoplamento entre os componentes do sistema, facilitando a manutenção e evolução do *software*.

Aplicar os princípios S.O.L.I.D no desenvolvimento de *software* contribui para a criação de sistemas modulares, flexíveis e de fácil manutenção. Eles ajudam a evitar código rígido e frágil, que é difícil de alterar sem causar efeitos colaterais indesejados.

No contexto deste projeto, a adoção conjunta dos princípios S.O.L.I.D e da Arquitetura Limpa é essencial para garantir que o *software* de gestão da informação seja escalável e adaptável às necessidades específicas dos usuários. Esse combo de técnicas permite estruturar o código de forma modular e independente, facilitando a implementação de novas funcionalidades e a manutenção do sistema ao longo do tempo. Assim, beneficia tanto os usuários finais quanto a equipe de desenvolvimento, permitindo atingir o objetivo final do manuscrito.

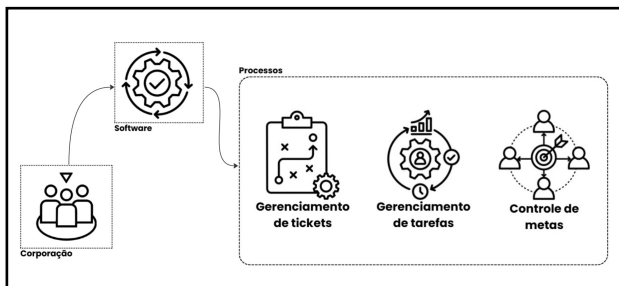
3 Metodologia

O desenvolvimento do sistema seguiu uma sequência metodológica baseada em quatro etapas principais: *i)* realizou-se uma revisão bibliográfica para embasamento teórico sobre arquitetura de *software* e sistemas de informação flexíveis; *ii)* procedeu-se à modelagem e ao projeto da arquitetura da solução, respeitando os princípios da Arquitetura Limpa e priorizando a modularidade; *iii)* desenvolveu-se o *backend*; e *iv)* desenvolveu-se o *frontend*, com foco em usabilidade e comunicação eficiente com as APIs previamente definidas.

3.1 Modelagem de Dados do Sistema

Como parte da segunda etapa metodológica, a modelagem de dados do sistema foi projetada para superar a rigidez de esquemas pré-definidos. O núcleo do modelo reside na capacidade de o usuário construir dinamicamente suas próprias tabelas, definindo colunas e tipos de dados conforme a necessidade específica de cada processo. A Figura 3 apresenta o modelo abstrato que viabiliza essa funcionalidade, permitindo a criação de estruturas para contextos diversos, como o registro de tickets ou o acompanhamento de metas de desempenho.

Figura 3: Visão geral do sistema como hub de informações

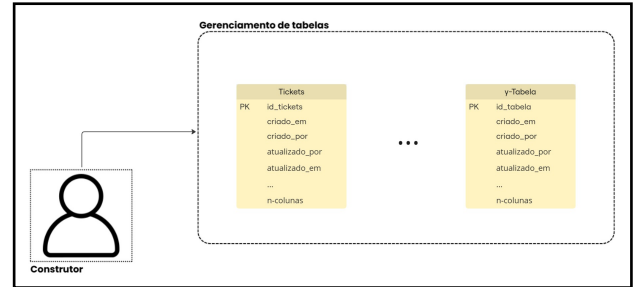


Fonte: Autoral

Como mostrado na Figura 4, a construção de

tabelas personalizadas está no centro da proposta. Cada usuário pode definir as colunas e os campos conforme o contexto de uso — como em módulos de registro de tickets ou metas de desempenho. Essa estrutura modular garante que alterações de escopo não afetem o sistema como um todo, preservando sua estabilidade e facilitando futuras integrações com sistemas externos.

Figura 4: Modelo abstrato de construção de bases de dados



Fonte: Autoral

Com isso, o projeto se posiciona como uma solução orientada à adaptabilidade, promovendo maior controle sobre os dados e respeitando a individualidade operacional de cada organização.

3.2 Tecnologias Utilizadas

O sistema foi implementado em Python, utilizando o *microframework* Flask para expor a API REST e o banco de dados PostgreSQL como mecanismo de persistência. Testes de integração foram conduzidos via *software* de requisições HTTP. Cada tecnologia foi escolhida por sua ampla adoção e por não introduzir acoplamento estrutural: todas as interações externas são mediadas por interfaces, permitindo a substituição futura de qualquer componente sem impacto na lógica de negócio.

Tabela 1: Principais tecnologias empregadas

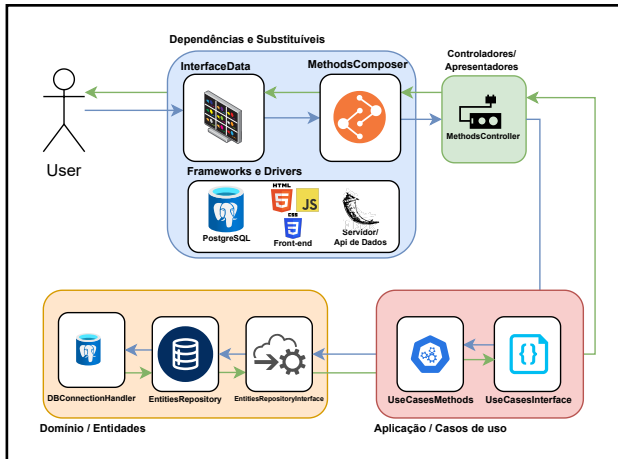
Componente	Papel	Motivação / Resiliência
Python	Linguagem base	Sintaxe clara, amplo ecossistema, forte suporte a testes.
Flask	Framework web	Estrutura minimalista; fácil troca por FastAPI, Django, etc.
PostgreSQL	Banco de dados	Confiável, aderência a SQL-ANSI; pode ser substituído por NoSQL via ORM.

3.3 Arquitetura da Solução

O código-fonte organiza-se em quatro camadas concêntricas, em conformidade com a Arquitetura

Limpa: Domínio e Entidades, Aplicação e Casos de Uso, Controladores e Apresentadores e Dependências e Substituíveis. As dependências apontam sempre para o centro, preservando as regras de negócio de detalhes tecnológicos. A Figura 5 apresenta o diagrama de alto nível da aplicação.

Figura 5: Diagrama de alto nível da aplicação



Fonte: Autoral

- **Entidades** Agrupam as regras de negócio imutáveis (*User*, *Table* etc.), sem dependências externas.
- **Casos de Uso** Orquestram operações do sistema (p. ex. inserir usuário, listar tabelas), consumindo apenas interfaces de repositório; mudanças de lógica concentram-se aqui.
- **Entidades** Convertem dados entre o formato externo (JSON/HTTP) e objetos internos e implementam repositórios via *Object Relational Mapping* (ORM) ¹. Um *adapter* genérico isola o *microframework* Flask, permitindo troca por qualquer servidor Web.
- **Interfaces Externas** Compreendem pontos de entrada/saída visíveis ao usuário ou a outros sistemas: servidor HTTP, *front-end* e *drivers* concretos. Credenciais de banco de dados e detalhes de infraestrutura residem em arquivos de configuração dessa camada ou nos módulos de infraestrutura. A eventual substituição do SGBD (Sistema Gerenciador de Banco de Dados) exige apenas ajuste de configurações e, no máximo, da implementação de repositório no nível de adaptadores, sem afetar as camadas internas.

A Figura 5 apresenta o fluxo de alto nível da execução do sistema, estruturado de acordo com os princípios da Arquitetura Limpa. A interação inicia-se com o envio de uma requisição externa (via *front-end*

¹ORM (Object-Relational Mapping) é uma técnica que permite interagir com bancos relacionais usando objetos da linguagem de programação.

ou cliente HTTP), que é recebida por uma rota do servidor Flask. Essa requisição é adaptada internamente por um módulo intermediário, que converte o formato externo (JSON/HTTP) em uma estrutura compreensível pelo núcleo da aplicação conforme ilustrado em 6.

Figura 6: Adaptador de Requisições.



Fonte: Autoral

A partir desse ponto, a requisição percorre as camadas internas do sistema: controladores validam os dados e invocam casos de uso específicos, os quais operam sobre entidades de domínio e interagem com repositórios desacoplados da infraestrutura. Ao final do fluxo, os dados são processados e retornam ao usuário no formato apropriado.

3.4 BackEnd

Para refletir essa arquitetura na estrutura do projeto, a organização dos diretórios foi cuidadosamente planejada. A Figura 7 apresenta a estrutura de diretórios resultante da aplicação desenvolvida:

3.4.1 Camada de Infraestrutura

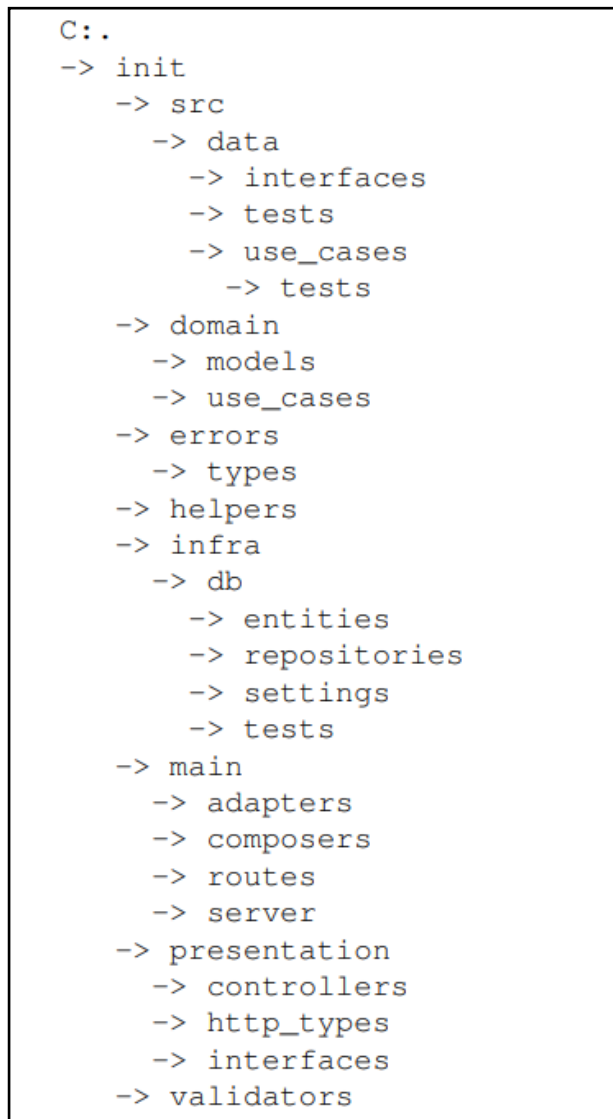
A camada de infraestrutura é responsável por lidar com as operações técnicas de baixo nível, como o acesso ao banco de dados e a configuração de conexões. No projeto desenvolvido, essa camada encontra-se no diretório *infra/* e contém componentes que se comunicam diretamente com o sistema de persistência (PostgreSQL), respeitando as interfaces definidas nas camadas superiores.

O módulo *DBConnectionHandler* centraliza a criação e o gerenciamento de sessões com o banco de dados por meio do software ORM SQLAlchemy. Essa abordagem permite abstrair o mecanismo de conexão com bancos relacionais, facilitando sua substituição por outros SGBDs compatíveis com SQL.

Os repositórios, localizados em *infra/db/repositories*, implementam as interfaces definidas em *domain/interfaces*. Eles encapsulam toda a lógica de persistência e recuperação de dados, garantindo que as demais camadas não dependam diretamente de comandos SQL ou do ORM.

As entidades persistentes do sistema, como por exemplo *Users* e *Tables*, são mapeadas via ORM no diretório *infra/db/entities*. Cada entidade reflete uma tabela do banco de dados e segue convenções de serialização e integridade dos dados definidas no domínio. Por meio dessa estrutura, a camada de infra

Figura 7: Estrutura de diretórios.



Fonte: Autoral

cumprir o papel de fornecer acesso seguro e controlado à base de dados, mantendo o núcleo do sistema completamente desacoplado da infraestrutura real.

3.4.2 Camada de Aplicação

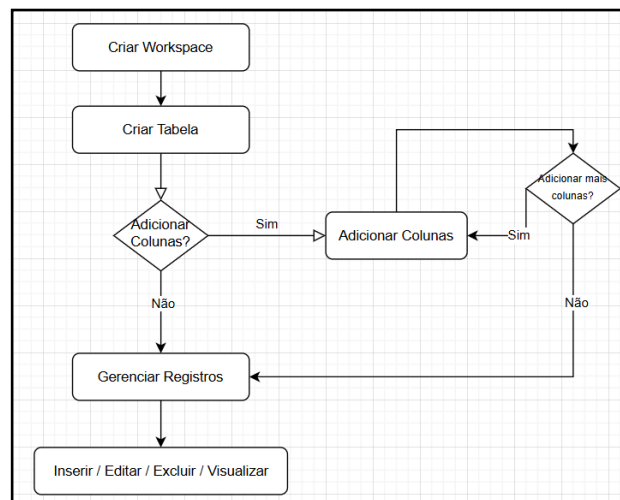
A camada de aplicação é responsável por coordenar as regras de negócio da solução. Nela residem os casos de uso, que encapsulam operações específicas da aplicação e funcionam como ponto de integração entre a lógica de domínio e os adaptadores externos. Cada caso de uso depende exclusivamente de abstrações — interfaces de repositório —, permitindo que a lógica central permaneça independente da infraestrutura, como banco de dados ou *framework* web.

Entre os casos de uso desenvolvidos, destacam-se especialmente aqueles que representam o núcleo funcional do sistema: a criação dinâmica de tabelas, a adição incremental de colunas personalizadas e as operações de manipulação de dados. Esses mecanismos

são os responsáveis por garantir que cada organização consiga estruturar suas próprias bases de dados, adaptando o sistema a diferentes contextos de uso sem reprogramação.

Após a estruturação dos principais casos de uso, foi elaborado um fluxo generalista para ilustrar as etapas principais que o sistema percorre no gerenciamento de tabelas e registros dinâmicos (Figura 8).

Figura 8: Fluxo generalista dos casos de uso do sistema



Fonte: Autoral

Inicialmente, o processo se inicia com a criação de um novo workspace, seguido da definição da primeira tabela. Em seguida, o sistema oferece a possibilidade de adicionar novas colunas, permitindo a personalização da estrutura conforme a necessidade do usuário. Esse procedimento é iterativo, possibilitando que múltiplas colunas sejam adicionadas antes da transição para a fase de gestão de registros.

Uma vez concluída a definição estrutural, o sistema habilita operações de manipulação de dados — como inserção, edição, exclusão e visualização de registros — sobre a tabela criada. Esse fluxo evidencia a flexibilidade e o caráter adaptativo da solução, possibilitando a construção incremental de bases de dados sob demanda, sem a necessidade de alterações no código-fonte.

3.4.3 Camada de Controladores e Apresentadores

A camada de Controladores e Apresentadores é responsável por receber as requisições externas, validar os dados, estruturar os objetos necessários para os casos de uso e, posteriormente, formatar as respostas que serão retornadas aos adaptadores de interface. Essa camada atua como intermediária entre o mundo externo (HTTP, protocolos, entrada de dados) e o núcleo interno da aplicação.

Sua principal responsabilidade é adaptar informações — tanto de entrada quanto de saída — para que o núcleo da aplicação permaneça protegido de varia-

ções tecnológicas, respeitando o princípio da inversão de dependências.

Em sistemas tradicionais baseados em *frameworks*, é comum que as requisições HTTP sejam tratadas diretamente nas camadas de negócio, o que cria um acoplamento forte entre o framework e a lógica da aplicação. Para evitar esse problema e manter a aderência aos princípios da Arquitetura Limpa, o projeto implementa uma abordagem baseada em adaptação explícita da requisição.

Figura 9: Adaptador de requisição

```
def request_adapter(
    request: FlaskRequest,
    controller: Callable
) -> HttpResponse:

    body = None
    if request.data:
        body = request.json

    http_request = HttpRequest(
        body=body,
        headers=request.headers,
        query_params=request.args,
        path_params=request.view_args,
        url=request.full_path
    )

    http_response = controller(http_request)
    return http_response
```

Fonte: Autoral

A função apresentada na Figura 9, realiza a conversão da requisição nativa do Flask em uma estrutura interna chamada `HttpRequest`, definida no domínio da aplicação. Essa conversão inclui o mapeamento do corpo da requisição dos cabeçalhos, dos parâmetros de consulta, dos parâmetros de caminho e da URL completa da chamada.

Após a adaptação, o objeto `HttpRequest` é repassado ao controlador apropriado, que manipula a lógica de orquestração de casos de uso, sem conhecer detalhes do protocolo HTTP ou do *framework* subjacente. Ao final do processamento, o controlador retorna um objeto `HttpResponse`, também independente de infraestrutura.

Essa estratégia representa um dos principais diferenciais arquiteturais do sistema, pois garante a completa independência entre o núcleo da aplicação e o *framework* web utilizado. Assim, caso futuramente se deseje substituir o Flask por outro *framework* (como FastAPI, Django ou mesmo uma API baseada em WebSockets), bastaria reimplementar o adaptador de requisição sem necessidade de alterações nas camadas internas.

3.4.4 Camada de Frameworks e Drivers

A camada *Frameworks & Drivers* representa o anel mais externo da Arquitetura Limpa. Seu papel é usar o núcleo da aplicação, nunca o contrário; por

isso, qualquer dependência presente nesta camada não “vaza” para dentro das demais.

No projeto, toda a infraestrutura de execução encontra-se no diretório `main/`, organizado conforme apresentado na Figura 10. Esses módulos funcionam como a “cola” que conecta o universo externo ao restante do sistema:

Figura 10: Estrutura da camada *Frameworks & Drivers*.



Fonte: Autoral

Ao receber uma requisição HTTP o sistema executa apenas três passos: (i) Adaptar a requisição do framework para um `HttpRequest` interno; (ii) Delegar ao controlador já configurado pelo *composer*; (iii) Devolver um `HttpResponse` convertido para o formato do framework.

Esse fluxo garante que troca de framework (por exemplo, de Flask para FastAPI) demande apenas a substituição dos adaptadores, preservando as demais camadas intactas.

Benefícios

- **Portabilidade** — mudanças de servidor, fila ou banco restringem-se a esta borda;
- **Testabilidade** — casos de uso e domínio podem ser exercitados sem subir infraestrutura;
- **Observabilidade e Segurança** — logging, métricas e autenticação permanecem fora da lógica de negócio.

Com isso, conclui-se a representação de todas as camadas da Arquitetura Limpa no projeto, assegurando independência tecnológica e facilidade de manutenção.

3.5 FrontEnd

A camada visual do sistema foi desenvolvida como uma prova de conceito para validar visualmente as funcionalidades do *backend* e materializar a proposta de valor do *software*. Em conformidade com os princípios da Arquitetura Limpa, o *frontend* posiciona-se na camada mais externa (*Frameworks & Drivers*), atuando como um agente externo que consome os serviços da aplicação, sem qualquer acoplamento com as regras de negócio ou a lógica de domínio.

A estratégia de implementação no servidor Flask baseou-se na utilização de *Blueprints*² para garantir

²No framework Flask, um *Blueprint* é um componente para organizar uma aplicação em partes menores e modulares. Cada *Blueprint* define um conjunto de rotas e lógicas relacionadas, promovendo a separação de responsabilidades.

uma clara separação de responsabilidades. O sistema foi segmentado em dois componentes principais:

- i. **Blueprint api:** Responsável exclusivamente por expor os endpoints da API REST, servindo como a interface de comunicação para qualquer cliente externo.
- ii. **Blueprint app_ui:** Responsável por renderizar a interface do usuário. Este componente atua como um cliente da própria API, realizando requisições HTTP para o **Blueprint api** a fim de buscar dados e executar operações. As páginas são renderizadas no lado do servidor (*Server-Side Rendering*)³ utilizando o motor de templates Jinja2.

Essa abordagem assegura que o *frontend* não acesse diretamente os casos de uso ou os repositórios, reforçando o desacoplamento. A comunicação via HTTP simula o comportamento de uma aplicação cliente totalmente independente, provando que o núcleo do sistema pode servir a múltiplos tipos de interface sem qualquer modificação.

Para validar a proposta central do projeto, o *frontend* implementou as seguintes funcionalidades essenciais, que espelham os principais casos de uso do sistema:

- i. **Gerenciamento de Workspaces:** Permite ao usuário criar, visualizar e organizar seus espaços de trabalho, que funcionam como contêineres para as bases de dados.
- ii. **Gerenciamento de Tabelas:** Dentro de cada *workspace*, o usuário pode gerenciar tabelas, que são as estruturas centrais de armazenamento de informação.
- iii. **Personalização de Estrutura e Dados:** A funcionalidade principal do software é materializada na interface, onde o usuário pode dinamicamente adicionar novas colunas a uma tabela existente e realizar operações completas de CRUD (*Create, Read, Update, Delete*) sobre os registros.

Dessa forma, a implementação do *frontend* não apenas oferece uma interface utilizável para o sistema, mas também serve como uma validação prática e tangível da flexibilidade e do desacoplamento arquitetural propostos neste trabalho.

4 Resultados

Esta seção apresenta os resultados obtidos, com foco na validação da arquitetura proposta e na demonstração funcional do sistema. A discussão é estruturada para primeiro comprovar o sucesso da implementação dos princípios de design e, em seguida,

³Abreviação para *Server-Side Rendering* (Renderização no Lado do Servidor). Nesta abordagem, o servidor web é responsável por gerar a página HTML completa, incluindo os dados dinâmicos, e enviá-la pronta para o navegador. Isso contrasta com o *Client-Side Rendering*, onde o navegador recebe um esqueleto de página e usa JavaScript para buscar dados e construir a interface.

ilustrar como esses princípios resultaram em um software funcional e flexível.

4.1 Validação da Arquitetura e Aplicação dos Princípios S.O.L.I.D.

O principal resultado deste trabalho é a materialização de um sistema de gestão da informação construído sobre os pilares da Arquitetura Limpa e dos princípios S.O.L.I.D. A validação dessa abordagem não se limita à funcionalidade do produto final, mas reside na própria estrutura do código-fonte, que garante manutenibilidade, testabilidade e baixo acoplamento.

Independência de Frameworks e Desacoplamento: Conforme detalhado na seção de metodologia, a independência de agentes externos foi alcançada. A utilização de um adaptador de requisição (ilustrado na Figura 9) que traduz o objeto request nativo do Flask para um objeto de domínio `HttpRequest`, representa um ponto central de desacoplamento. Essa camada de tradução assegura que os controladores e os casos de uso operem sobre abstrações, totalmente alheios ao protocolo HTTP ou ao framework web. A consequência prática é que a substituição do Flask por outro framework, como FastAPI ou Django, exigiria apenas a reescrita desse adaptador, sem impacto algum nas camadas internas do sistema. Esse resultado valida diretamente o Princípio da Inversão de Dependência (D), onde os módulos de alto nível (casos de uso) dependem de abstrações (`HttpRequest`, `HttpResponse`) e não de detalhes de baixo nível do framework.

Aplicação dos Princípios S.O.L.I.D. na Prática: Os princípios S.O.L.I.D. foram aplicados de forma consistente em todo o desenvolvimento, resultando em um código modular e coeso:

- i. **Princípio da Responsabilidade Única (S):** Cada caso de uso (ex: `AddColumn`, `CreateTable`) encapsula uma única funcionalidade do sistema. Da mesma forma, os repositórios (ex: `TableRepository`) têm a única responsabilidade de interagir com a fonte de dados para uma entidade específica. Isso impede que classes se tornem "faz-tudo", simplificando a manutenção e os testes.
- ii. **Princípio Aberto/Fechado (O):** A arquitetura é inerentemente aberta à extensão. Para adicionar uma nova funcionalidade, como "Exportar Tabela para CSV", um novo caso de uso (`ExportTableUseCase`) pode ser criado sem modificar os casos de uso existentes de criação ou leitura de tabelas. A lógica de negócio é estendida, não alterada.
- iii. **Princípio da Substituição de Liskov (L):** Este princípio foi respeitado na implementação dos repositórios. A camada de caso de uso interage com uma interface de repositório (ex: `TableRepositoryInterface`). A implementação concreta (`TableRepository`), pode ser substituída por uma

implementação em memória para testes (InMemoryTableRepository) sem quebrar o contrato, pois ambas respeitam a mesma interface.

- iv. **Princípio da Segregação de Interface (I):** As interfaces de repositório foram projetadas para serem específicas e enxutas. Em vez de uma única interface monolítica IRepository com métodos de CRUD para todas as entidades, foram criadas interfaces segregadas (ex: UserRepositoryInterface, TableRepositoryInterface), garantindo que uma classe cliente não dependa de métodos que não utiliza.
- v. **Princípio da Inversão de Dependência (D):** Este princípio é, talvez, o mais evidente na arquitetura global do sistema. Conforme discutido no parágrafo de abertura desta seção, a estratégia de adaptação das requisições HTTP é a aplicação direta deste princípio garantindo que a lógica de negócio permaneça isolada de detalhes de infraestrutura.

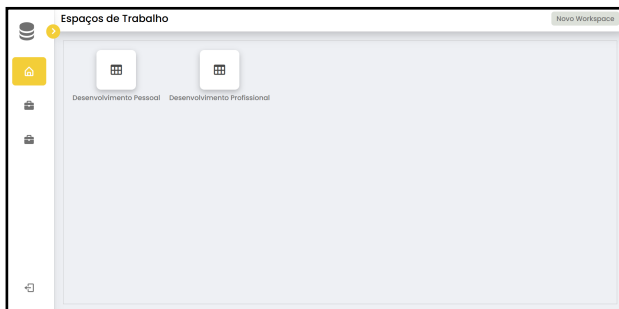
A validação arquitetural, portanto, é confirmada pela análise da estrutura do código e pela forma como ela habilita a evolução do sistema. O resultado é um backend robusto, testável em isolamento e resiliente a mudanças tecnológicas em suas camadas externas, atingindo o objetivo central deste trabalho.

4.2 Validação Funcional via Interface Gráfica

Como consequência da arquitetura, foi possível desenvolver uma interface de usuário que demonstra de forma prática a flexibilidade do sistema. A interface, atuando como um cliente da API, valida o fluxo de uso principal e materializa a proposta de valor do software para o usuário final.

A jornada do usuário começa na tela de espaços de trabalhos denominados Workspaces, conforme ilustrado na Figura 11. Cada workspace funciona como um contêiner isolado, permitindo que diferentes projetos ou equipes organizem suas informações de forma independente.

Figura 11: Tela Inicial de Espaços de Trabalho.

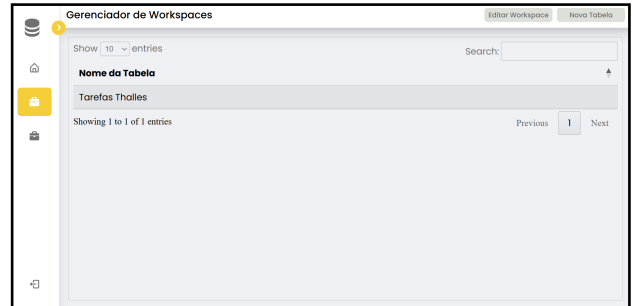


Fonte: Autoral

Ao selecionar um workspace, o usuário acessa sua área de gerenciamento (Figura 12), que centraliza as

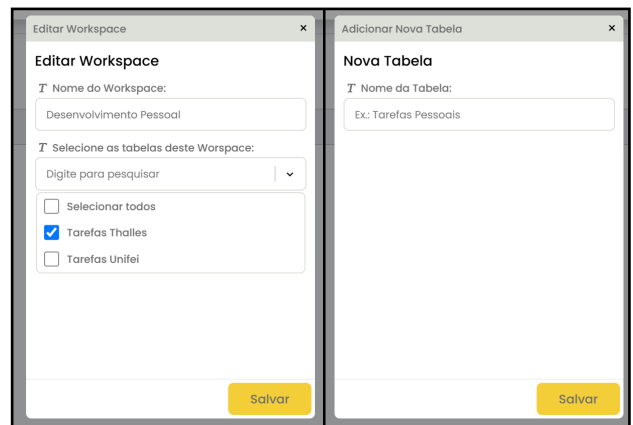
tabelas associadas e oferece ações-chave, como a adição de novas tabelas. A interatividade da página é exemplificada na Figura 13, que exibe os modais correspondentes às funcionalidades de gerenciamento, demonstrando a capacidade do sistema de guiar o usuário através de operações complexas de forma intuitiva.

Figura 12: Espaço de Trabalho.



Fonte: Autoral

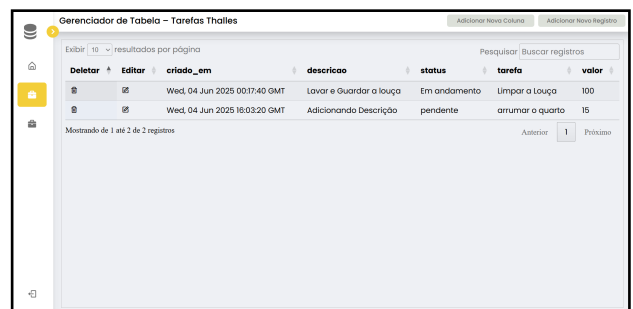
Figura 13: Modais de Funcionalidades da Página.



Fonte: Autoral

O fluxo prossegue para o núcleo da gestão de dados. A Figura 14 apresenta a interface de uma tabela, onde sua estrutura e seus registros são exibidos.

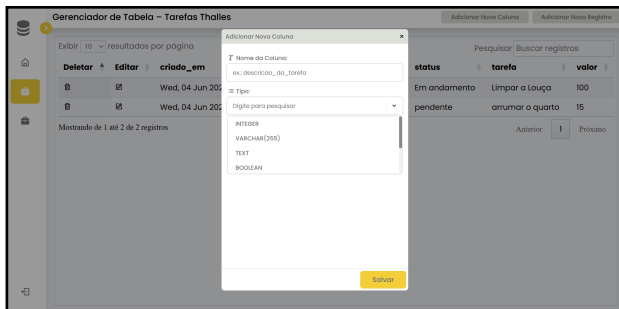
Figura 14: Interface de gerenciamento de tabelas associadas a um Workspace específico.



Fonte: Autoral

O principal diferencial do sistema é validado na prática através da funcionalidade de personalização da estrutura de dados, materializada na Figura 15. O usuário pode adicionar colunas dinamicamente, especificando seu nome e tipo. Este resultado comprova o atendimento ao requisito fundamental do projeto: oferecer uma ferramenta flexível que se molda ao processo do usuário, e não o contrário.

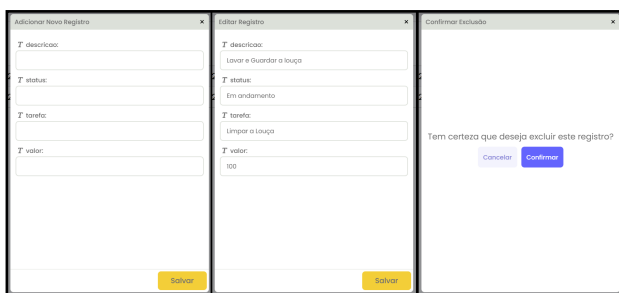
Figura 15: Demonstração da funcionalidade principal: adição dinâmica de colunas a uma tabela, permitindo a personalização completa da estrutura de dados.



Fonte: Autoral

Uma vez definida a estrutura, o sistema consolida-se como uma ferramenta funcional para a gestão da informação. A Figura 16 demonstra a interface de visualização e manipulação de registros, onde o usuário pode executar operações de CRUD, completando o ciclo de uso da base de dados personalizada.

Figura 16: Operações de CRUD em uma tabela com estrutura personalizada, evidenciando a aplicação prática do sistema no gerenciamento de informações.



Fonte: Autoral

Em conjunto, a sequência de validações apresentada nesta seção demonstra o cumprimento da proposta inicial do projeto em suas duas frentes. A validação arquitetural comprovou a robustez, manutenibilidade e o desacoplamento do backend, enquanto a validação funcional materializou esses benefícios em uma interface de usuário.

5 Conclusão

Este trabalho apresentou o desenvolvimento e a validação de um software de gestão da informação, cuja arquitetura foi concebida para permitir personalização de bases de dados por parte do usuário final. A investigação centrou-se na aplicação dos princípios da Arquitetura Limpa e do S.O.L.I.D. como pilares para alcançar flexibilidade e manutenibilidade em sistemas de informação.

A arquitetura resultante demonstrou a viabilidade de desacoplar as regras de negócio centrais de detalhes de infraestrutura, como o framework web e o sistema de persistência de dados. A utilização de adaptadores e a estrita adesão ao princípio da inversão de dependência foram fundamentais para garantir que o núcleo do sistema permaneça agnóstico a tecnologias externas, facilitando futuras evoluções e substituições de componentes.

O estudo de caso funcional, materializado através de uma interface de usuário, validou a principal proposta de valor: a capacidade do usuário de criar e gerenciar dinamicamente suas próprias estruturas de dados. Este resultado prático corrobora a tese de que uma arquitetura bem estruturada não apenas melhora a qualidade técnica do código, mas também habilita a construção de funcionalidades complexas e adaptáveis.

Conclui-se, portanto, que a adoção de paradigmas como a Arquitetura Limpa é um fator determinante para o desenvolvimento de sistemas de software que respondam eficazmente às necessidades dinâmicas das organizações, promovendo a criação de soluções robustas, escaláveis e sustentáveis a longo prazo. Trabalhos futuros podem se concentrar na expansão das capacidades de integração com outras plataformas e na implementação de mecanismos de análise de dados sobre as estruturas personalizadas.

Agradecimentos

Agradeço, primeiramente, ao Professor Rodrigo A. S. Braga, pela orientação fundamental durante a primeira etapa deste trabalho (TCC1), que estabeleceu os alicerces do projeto, e por sua valiosa participação na banca de defesa final.

Ao Professor Eduardo, expresse minha profunda gratidão por ter assumido a orientação na etapa conclusiva (TCC2). Sua condução, apoio e insights foram indispensáveis para a maturação e finalização da pesquisa. Agradeço, igualmente, por sua importante contribuição como membro da banca no TCC1, cujos apontamentos foram cruciais para o direcionamento do trabalho.

Estendo meus agradecimentos ao Professor Carlos, por sua disponibilidade e por suas pertinentes contribuições como membro de ambas as bancas examinadoras. Suas análises críticas e sugestões foram essenciais para o aprimoramento da qualidade deste manuscrito.

Referências

- Erich Gamma, Richard Helm, R. J. D. J. V. (1994). Design patterns: Elements of reusable object-oriented software.
- Guilherme Biff Zarelli (2020). Descomplicando a clean architecture. Acesso: 09/11/2024.
- M. E. Porém; V. C. Bissoli; R. C. Belluzzo (2012). Vantagem competitiva nas empresas contemporâneas: a informação e a inteligência competitiva na tomada de decisões estratégicas. Acesso: 04/11/2024.
- Martin, R. C. (2009). Código limpo: Habilidades práticas do agile software.
- Martin, R. C. (2019). Arquitetura limpa: o guia do artesão para estrutura e design de software.
- McKnight, W. (2013). Information management: Strategies for gaining a competitive advantage with data.
- R. A. C. Júnior, T. S. Nunes (2021). O impacto da transformação Ágil na cultura organizacional: Das práticas e valores organizacionais a gestão da mudança. Acesso: 06/11/2024.
- R. R. Barbosa (2020). Gestão da informação e gestão do conhecimento: evolução e conexões. Acesso: 07/11/2024.
- Robert C. Martin (2012). The clean architecture. Acesso: 09/11/2024.
- Ugonna Thelma (2020). The s.o.l.i.d principles in pictures. Acesso: 09/11/2024.